

AI WeatherWise - Deliver intelligent weather forecasts and insights.

Description

AI WeatherWise is a powerful RESTful backend system developed using **Node.js** and **Express.js**, with **MongoDB** and **Mongoose ODM** ensuring efficient data storage and management. It provides **real-time weather analytics**, enhanced by **Google Gemini AI**, which generates intelligent summaries and personalized suggestions based on current environmental data.

The platform incorporates **secure JWT-based authentication**, **bcrypt-powered password encryption**, and a **comprehensive set of endpoints** for handling favorite locations and retrieving weather insights. It also includes a **robust fallback mechanism** that maintains API functionality even when external keys are unavailable, complemented by **centralized error management** and **request sanitization** for reliability and security.

Scenario-Based Case Study

Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather conditions and receive AI-driven activity suggestions based on current forecasts.

Problem

- Difficulty managing and tracking weather information for multiple favorite locations.
- No personalized recommendations for outdoor activities or suitable clothing.
- Weather data displayed only as raw numbers instead of easy-to-read summaries.
- Lack of a secure system to store and sync user preferences across devices.

Solution

AI WeatherWise provides a centralized platform for weather data collection and AI processing. It securely manages **User** and **Location** databases, enabling customized insights and automated recommendation workflows powered by **Google Gemini AI**.

Usage

- **User Functions:** Create, update, or delete favorite cities; access real-time weather data; and request AI-generated summaries and recommendations.
- **Developer/Admin Functions:** Monitor API performance, manage user registrations, and verify weather and AI service integrations.

- **Public Access:** Retrieve current weather metrics through open endpoints without requiring authentication..

Outcome

By leveraging AI and centralized data management, users spend less time deciphering raw metrics. The system ensures data integrity via Mongoose schemas and provides high-value, context-aware weather advice through Gemini integrations.

1. Software Requirements

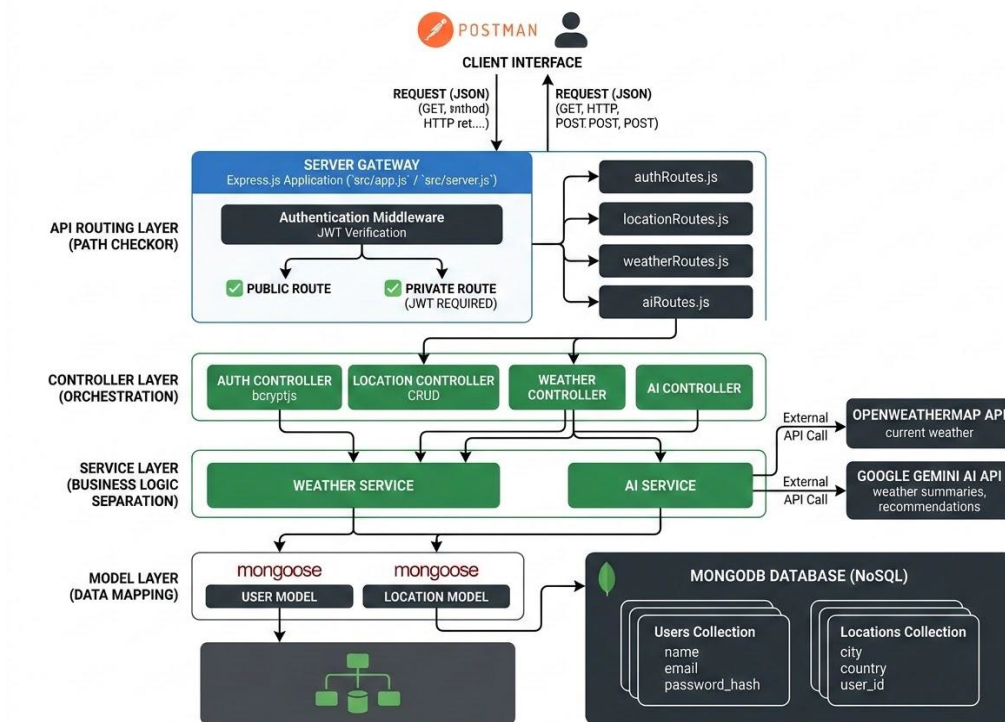
- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Gemini AI SDK:** Integration using @google/generative-ai for natural language insights.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Epic 1: PROJECT ARCHITECTURE

AI WeatherWise – System Architecture Overview



The **AI WeatherWise** application uses the **MVC (Model–View–Controller)** design pattern to separate business logic from data handling.

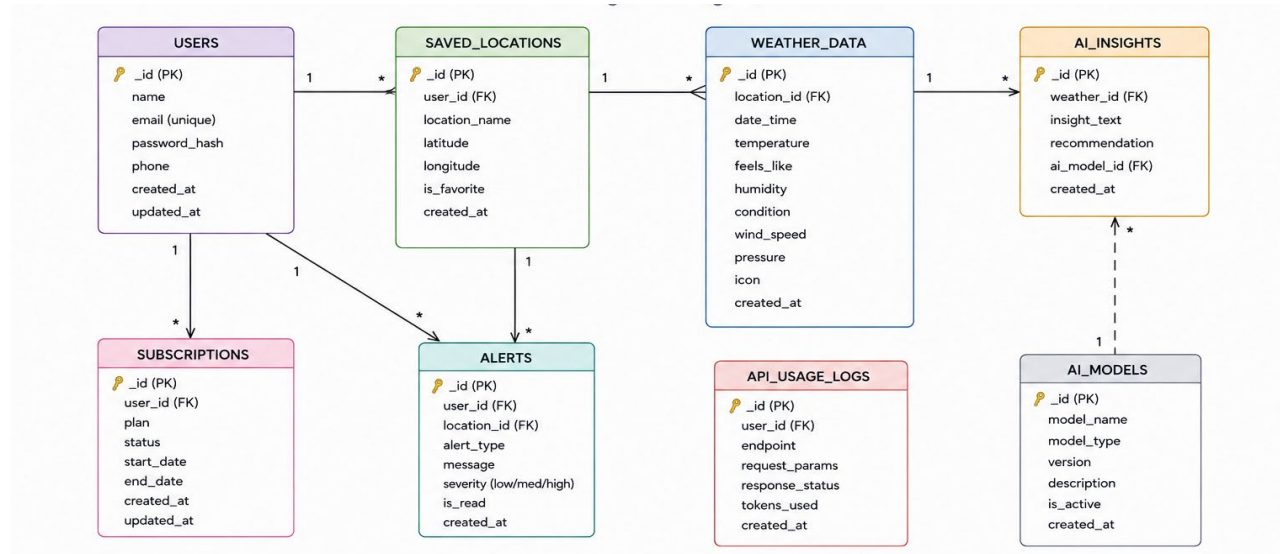
- The **Controller layer** manages weather data retrieval from **OpenWeatherMap** and generates AI insights using **Google Gemini**.
- The **Model layer** handles **User** and **Location** entities for structured data management.

Components Definition

- **Interface:** API tools such as **Postman** are used to test different route paths and retrieve JSON responses.
- **Express Server Gateway:** Listens on network ports, defines HTTP methods (**GET**, **POST**, **PUT**, **DELETE**), processes JSON requests, and manages **CORS** headers.
- **Authentication & Role Middleware:** Extracts tokens from request headers, verifies user identity, and checks if the user's role has permission to access protected routes.
- **Service Modules (Logic Layers):** Apply specific business rules for blog operations (Draft, Pending Approval, Scheduled, Published), comment handling, and system reports.
- **Database Layer:** **Mongoose** converts application logic into optimized **MongoDB** queries for efficient data access.

- **AI Integration Layer:** Uses **@google/generative-ai** to create weather summaries and personalized recommendations based on real-time temperature and conditions.

Entity-Relationship (ER) Diagram Description



The data architecture manages User profiles and their associated favorite Locations through logical references:

1. User Entity

- **_id** (Primary Key - ObjectId)
- **name** (String, Required)
- **email** (String, Required, Unique)
- **password** (String, Hashed, Required)
- **role** (String, Default: "reader")

2. Location Entity

- **_id** (Primary Key - ObjectId)
- **user** (Foreign Key referencing User Collection)
- **city** (String, Required)
- **country** (String, Required)
- **createdAt** (Date, Default: Now)

Key Structural Mappings

- **User to Location (One-to-Many):** A registered user can save multiple favorite locations.

Key Features

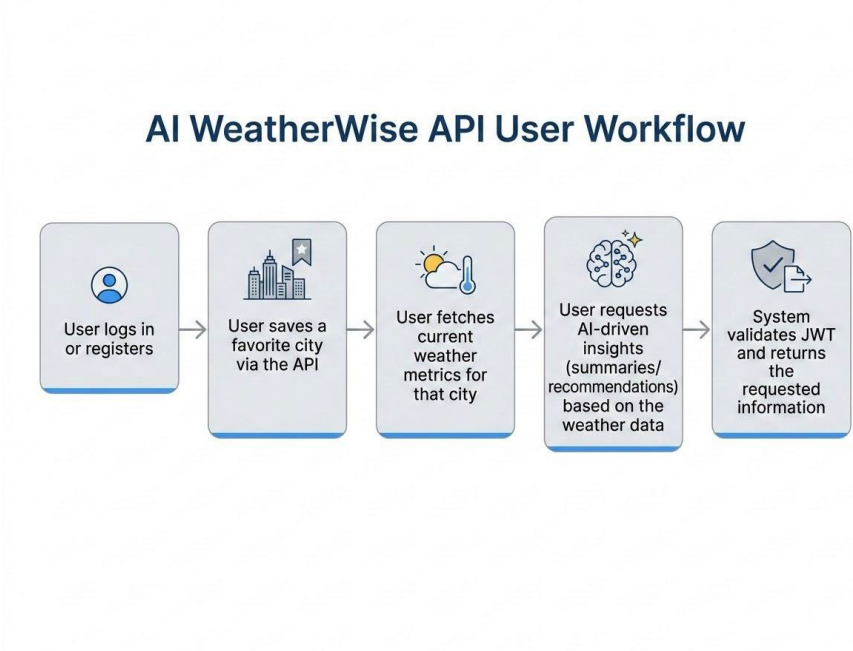
- **Role-Based Route Protection:** Secure token handling ensures that users without proper authorization cannot change key system configurations.
- **Blog Management Engine (CRUD):** Provides full control over blog content, including schema setup, status updates, and media attachments.
- **Categorization Framework:** Built-in tag indexing supports efficient filtering and searching, improving content visibility and organization
- **Comment Moderation System:** Protected endpoints allow administrators to update comment statuses such as approved, pending, or spam.
- **Data Sanitization and Security:** Cleans and validates input data to prevent **Cross-Site Scripting (XSS)** and **NoSQL injection** vulnerabilities.

Roles and Responsibilities

- **Admin:**
 - Responsible for user account management, including creating, deleting, or suspending accounts.
 - Oversight of API health and system logs to monitor performance.
 - Monitoring and managing database integrity.
- **Registered User:**
 - Responsible for creating and maintaining a list of favorite locations.
 - Initiates requests to fetch real-time weather data.
 - Interacts with the **Gemini AI** service to receive personalized weather summaries and activity recommendations.
 - Manages authentication credentials, including updating personal profile details..

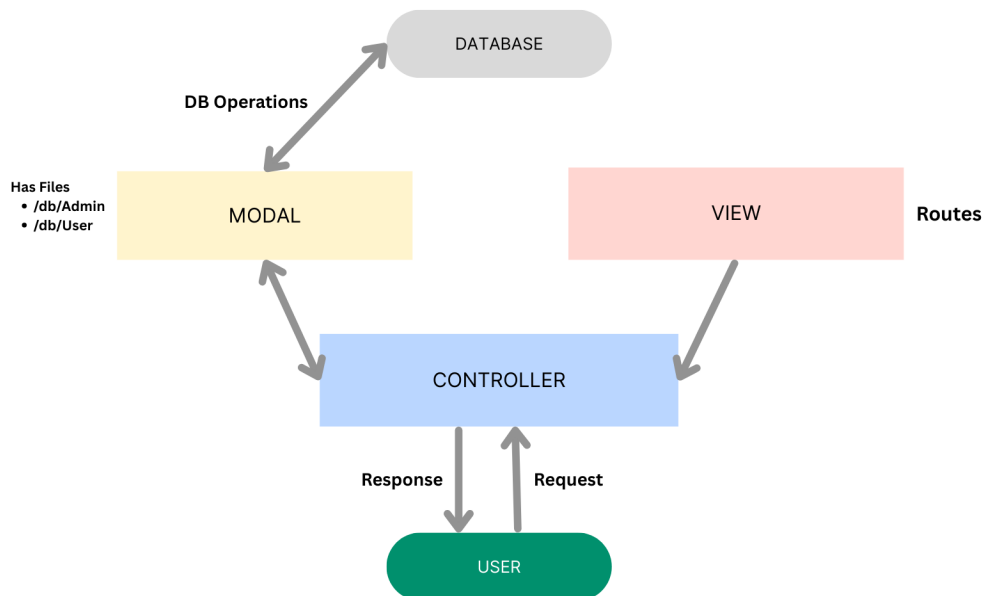
User Flow:

1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.
5. System validates JWT and returns the requested information.



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines the data structure using **Mongoose schemas**, which map specific fields to collections in **MongoDB**. This layer sets default values, indexing rules, and transformation parameters before data validation and storage occur.

Controller Layer (Execution Matrix)

Acts as the central processing unit of the system. It receives requests from defined routes, validates their structure, delegates tasks to the model layer, and returns standardized **JSON** responses after processing.

View Layer (Routing Interface)

Since this is a **headless backend service**, there are no traditional user interface templates. Instead, the view layer functions as an **API routing interface**, connecting network requests on specific paths directly to their corresponding controller logic.

Epic 2: Project Setup and Configuration

Creating the Project Folder

- Create a new folder named **AI Weatherwise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer  
@google/genai
```

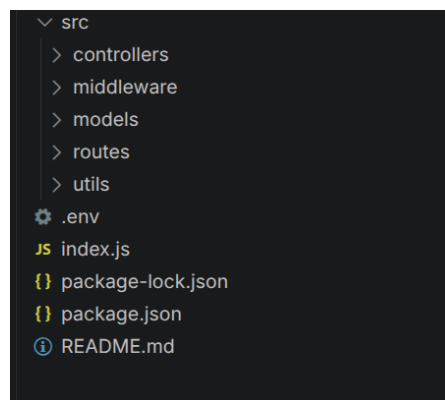
Install Nodemon for development:

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

Create a **.env** file in the project root and configure the required environment variables.

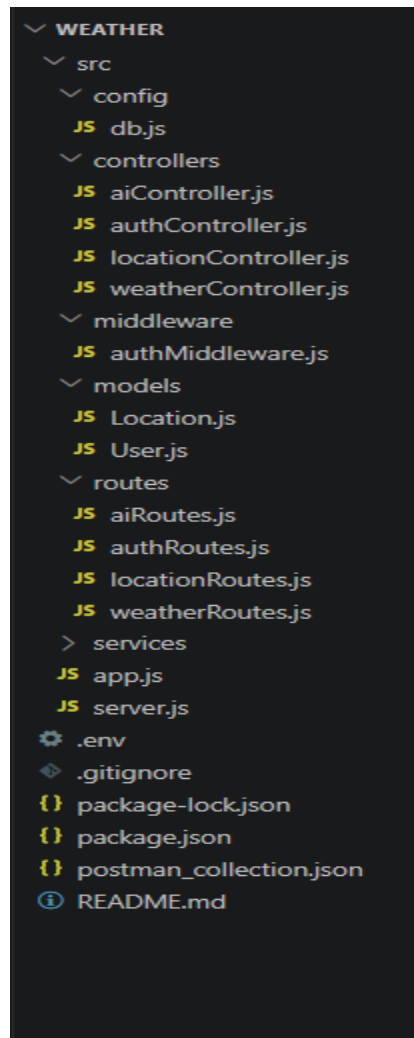
```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

Epic 3: Backend Development

The AI weatherwise backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.

Backend

Structure:



Core System Script Implementations

1. Application Entry Point (**index.js**)

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```

const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Epic4: Database Configuration

API Testing Demo :

1. Configure MongoDB:

- Install Mongoose.
In your Node.js project directory, run:
[npm install mongoose](#)
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

Location Schema:

```

const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;

```

User schema:

```

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\.w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});

```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Epic 5: Project Executions:

The **AI WeatherWise** backend can be run locally using **Node.js**. Before starting the application, ensure that **MongoDB** is active and all required **environment variables** are properly configured. The backend provides a set of **RESTful APIs**, which can be tested using tools such as **Postman** or **Thunder Client**.

Running the Backend Server

Install all required dependencies:

```
npm install
```

Start the development server:

```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API testing:

Register API:

The screenshot shows a REST client interface with the URL `http://localhost:5001/api/auth/register`. The request is a **POST** with a JSON body:

```
{  "name": "John Doe",  "email": "john@mail.com",  "password": "123456"}
```

The response status is **201 Created** with a time of 172 ms and a size of 556 B. The response body is a JSON object:

```
{  "success": true,  "data": {    "id": "6a4391c99e4426c279d7e4fb",    "name": "John Doe",    "email": "john@mail.com",    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhNDM5MmM5OUU0NDIyZiI3OWQ3ZTRhYiIsIm1hdCI6MTc4Mjg4MzEyOSwiZXhwIjoxNzg1MDA1MTI5fQ.7SKxk8zq11JPRb11lW1pR_EyGjddG-npRBYitCNeKk"  }}
```

Login API:

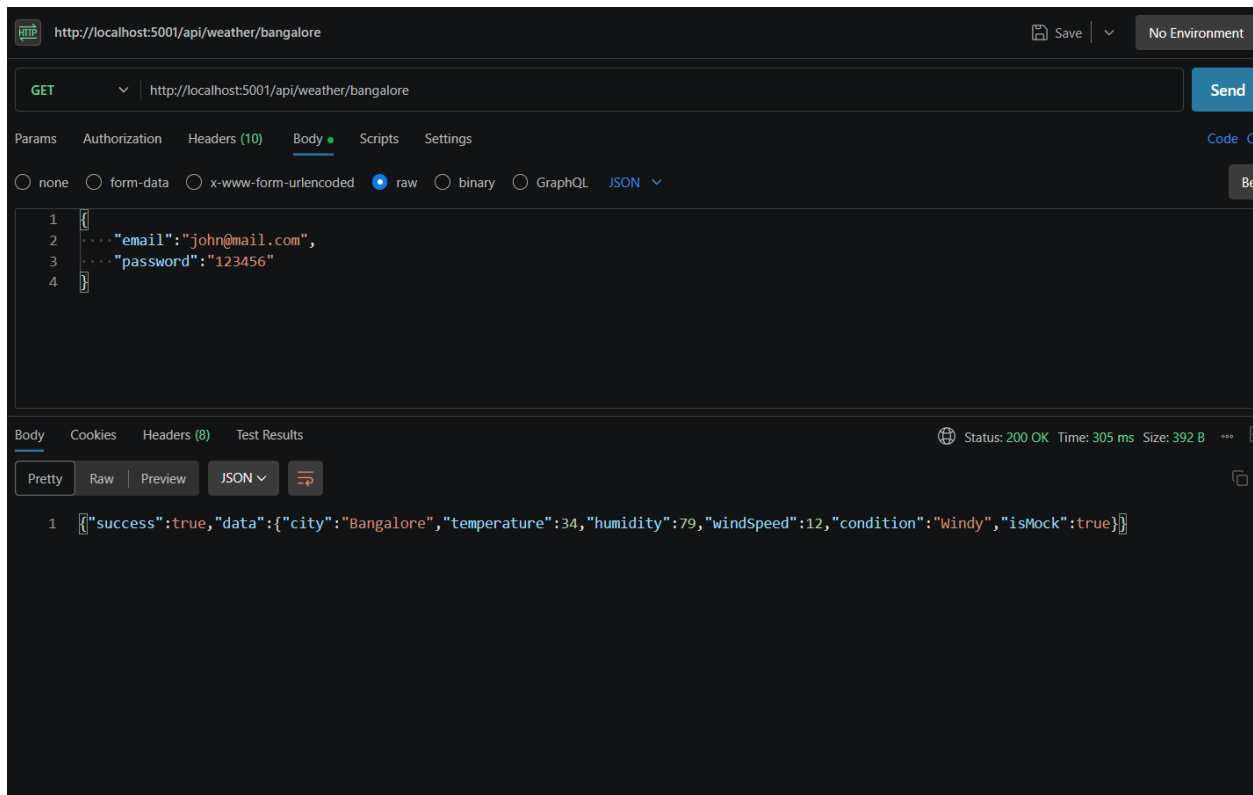
The screenshot shows a REST client interface with the URL `http://localhost:5001/api/auth/login`. The request is a **POST** with a JSON body:

```
{  "email": "john@mail.com",  "password": "123456"}
```

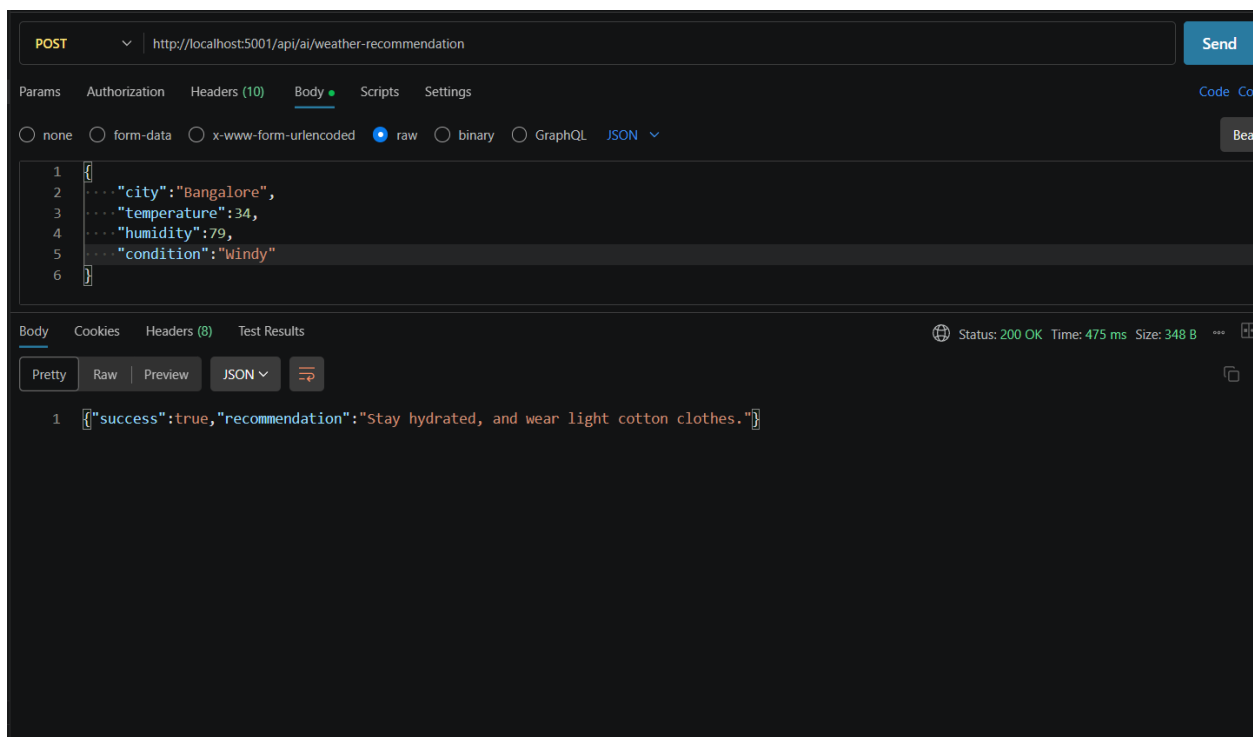
The response status is **200 OK** with a time of 90 ms and a size of 551 B. The response body is a JSON object:

```
{  "success": true,  "data": {    "id": "6a4391c99e4426c279d7e4fb",    "name": "John Doe",    "email": "john@mail.com",    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhNDM5MmM5OUU0NDIyZiI3OWQ3ZTRhYiIsIm1hdCI6MTc4Mjg4MzEyOSwiZXhwIjoxNzg1MDA1MTI5fQ.7SKxk8zq11JPRb11lW1pR_EyGjddG-npRBYitCNeKk"  }}
```

Weather API:



Recommendation API:



API testing Videolink: https://drive.google.com/file/d/1T9ISUC3i7RtjQG-NMM25LI3gFAFuQHUL/view?usp=drive_link

DemoVideoLink:https://drive.google.com/file/d/1yosa5qj16LNQeMnt9R1O-WpT08UEuF5u/view?usp=drive_link

CodeDrive:https://drive.google.com/drive/folders/1J1we0RBhTGDJgKC_egLqnSUuWsO06mXJ?usp=sharing